第六章 测试与评估：AI增强设计研究工具的验证方法

> **本章导言**
>
> 经过前五章的学习与开发，你已经将一个经典的设计研究方法重构为可运行的AI技能（Skill）。这个Skill能够处理数据、调用模型、输出分析结果，并且具备了基本的交互体验。然而，一个关键问题仍然悬而未决：这个Skill真的值得信任吗？它的输出是否准确、一致、高效？它与人类研究者协作时是否产生了真正的增益？这些问题的答案，不能依赖于开发者的主观感觉，而必须通过系统的、可重复的测试来获得。
>
> 本章聚焦于AI增强设计研究工具的测试与评估方法。我们将首先讨论测试AI工具与传统软件测试的本质差异，进而建立一个五维测试框架（准确性、一致性、效率、健壮性、体验），并引入“协同增益”这一额外的评估维度。随后，我们将详细阐述如何利用AI本身来辅助测试——从测试数据生成、参考答案构建，到自动化测试脚本编写和LLM-as-Judge裁判机制。最后，我们将完整呈现测试执行与数据分析的流程，并指导你撰写一份结构化的测试报告。
>
> 学完本章，你将不仅能够对自己的Skill进行严格的验证，更能够将这套测试方法论迁移到未来任何AI增强的设计研究工具开发中。测试不是项目的终点，而是持续改进的起点——测试报告中的每一项发现，都将直接转化为下一轮迭代的具体任务。

本章学习目标

完成本章学习后，读者应能够：

1. 区分测试传统软件与测试AI工具的本质差异，理解AI Skill测试的三个独特挑战；
2. 运用五维测试框架（准确性、一致性、效率、健壮性、体验）设计全面的测试方案；
3. 使用AI生成高质量的测试数据集，并构建参考答案与基线对比方案；
4. 编写自动化测试脚本，实现定量性能测试的批量运行与结果记录；
5. 设计和执行定性体验测试，收集SUS量表和半结构化访谈数据；
6. 运用LLM-as-Judge技术辅助评估生成式输出的质量；
7. 基于测试数据撰写结构化的测试报告，并将发现转化为改进计划。

6.1 设计研究AI工具的测试方法论

6.1.1 为什么测试AI Skill不同于测试传统软件

在第五章结束时，你的Skill已经完成了交互打磨，并通过中期同行评审获得了初步的外部反馈。但同行评审更多是定性的、探索性的，它能够揭示明显的体验问题，却无法量化地回答“这个Skill的准确率是多少”“它与纯人工相比效率提升了多少”“它在不同难度输入上的表现差异如何”这些问题。要回答这些问题，我们需要进入系统化的测试阶段。

然而，测试一个依赖AI的工具与测试传统软件有着本质区别。传统软件的测试建立在“确定性预期”的基础上——给定输入A，预期输出B。如果输出不是B，就是缺陷（bug）。测试用例可以精确地编写，预期结果可以明确地定义，回归测试可以自动运行。但在AI Skill中，这种确定性往往不复存在。

挑战一：没有唯一的“正确答案”。在传统软件中，2+2的正确答案是4，任何其他输出都是错误。但在设计研究Skill中，一段用户反馈的情感分析——AI判断为“中性”而研究者判断为“略微负面”——这算错误吗？从严格意义上，AI漏掉了微妙的负面信号，可以认为是不够准确；但从另一个角度看，这种“微妙到需要争论”的判断本身就缺乏共识，不同的人类研究者也可能给出不同答案。测试AI Skill时，许多输出落在灰色地带，需要建立“可接受的误差范围”而非绝对的正确/错误。

挑战二：输出具有随机性。如果你调用大模型API时设置了`temperature > 0`（即使是0.1），同样的输入两次调用可能得到措辞不同的输出。情感极性可能一致（都是“正面”），但描述的文字不同——这不构成错误。然而，如果情感极性两次不同（一次“正面”、一次“中性”），这就是一个值得关注的不一致。测试方案需要能够区分“可接受的变异”（措辞变化）和“不可接受的不一致”（实质性判断变化）。

挑战三：错误的模式往往是“偏差”而非“崩溃”。传统软件的缺陷通常导致程序崩溃、报错、异常终止——这些很容易被检测到。AI Skill的错误更为隐蔽：代码完美运行，界面毫无异常，所有输出都格式正确——但结论系统性地偏了。情感分析持续将轻微的负面表达误判为中性；聚类算法将语义上其实不太相似的内容分到了一起。这些“偏差”不会触发任何报错，只有通过对照参考答案或人类判断才能暴露。

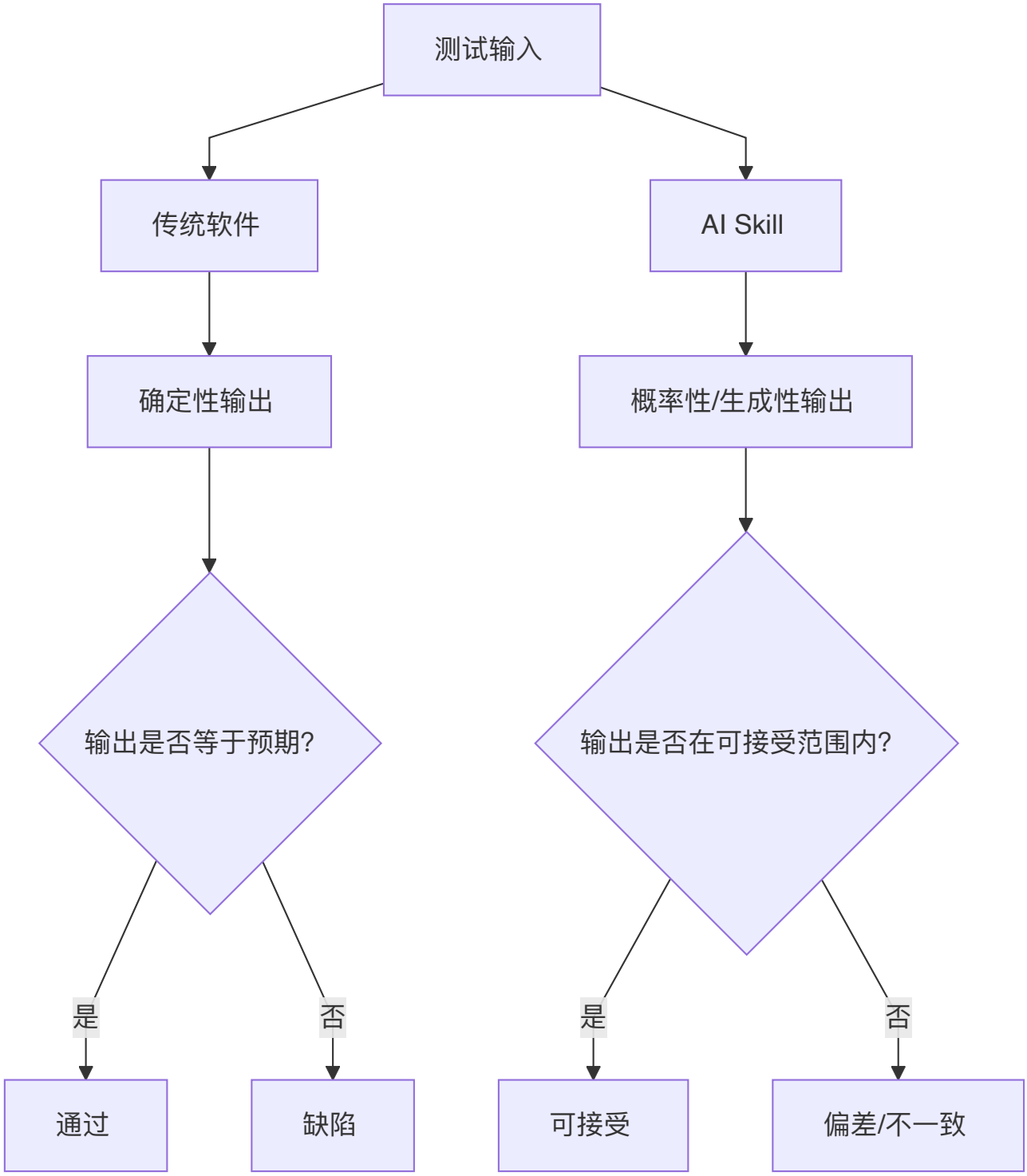


图6-1 传统软件测试与AI Skill测试的本质差异

因此，测试AI Skill需要一套全新的方法论——它必须同时兼容不确定性、处理模糊的正确答案边界、并能够检测系统性的偏差而非仅关注崩溃性错误。本章建立的方法论正是为此而设。

6.1.2 五维测试框架

在第3章中，我们建立了评估框架的五个维度（准确性、一致性、可解释性、效率、体验），用于指导Skill的设计质量。现在，当Skill已经建成并需要被测试时，我们将这五个维度转化为可执行的测试方法。注意：可解释性在测试阶段主要通过定性体验测试来评估（用户是否能理解AI的推理），因此在本章的定量测试框架中，我们将其融入体验维度，同时增加一个技术层面的"健壮性"维度，形成用于测试的五维框架。

维度	定义	测试方法	关键指标
准确性	输出与“正确输出”的匹配程度	人机比对、参考答案对照	准确率、召回率、F1、混淆矩阵
一致性	相同条件下输出的稳定性	重测、分半、边界输入测试	一致率、语义相似度、标准差
效率	完成任务的耗时与资源消耗	计时日志、基线对比	端到端耗时、人工介入时间、API调用次数
健壮性	异常输入下的行为表现	异常用例注入、压力测试	错误处理率、降级成功率
体验	用户的主观感受与信任度	SUS量表、半结构化访谈、行为观察	SUS总分、信任评分、困惑点数量

表6-1 五维测试框架

6.1.3 双轨测试：量化性能与协同增益

除了上述五个维度外，设计研究AI Skill还需要回答一个更高层次的问题：**人机协同是否真正产生了增益？** 这个问题无法通过单独测试Skill来回答，而需要比较三种条件下的输出质量：

- **条件A：AI单独**。只运行你的Skill，不经过人类修改，记录输出质量。
- **条件B：人类单独**。让一位设计研究者（最好是熟悉该方法但不参与开发的同学）用传统方式完成同样的任务，记录输出质量和耗时。
- **条件C：人+AI协同**。研究者使用你的Skill的完整交互版本（包含人工修正节点），记录最终输出质量和总耗时。

如果条件C的质量 > max(条件A, 条件B)，说明协同产生了**正增益**——人机搭档比任何一方单独工作都更好。如果条件C介于A和B之间，说明协同存在摩擦，但尚未完全抵消AI的优势。如果条件C < min(条件A, 条件B)，说明协同在拖后腿——这通常意味着人工节点设计不当，导致用户频繁打断有效的AI分析路径，或者用户在人工节点上做出了错误判断。

在测试方案中，建议至少进行一次小规模的协同增益对比实验（5-10条数据，1-2名被试），作为定性评估的补充。

6.2 测试方案的AI辅助设计

6.2.1 为什么让AI设计测试方案

设计一套完整的测试方案是一项繁重的工作。你需要穷尽正常和异常场景，需要设计覆盖多维度问题的测试数据，需要编写比对逻辑和统计脚本。这些任务的“模式化”程度很高——测试方案的结构在不同Skill之间是相对固定的，只是内容需要根据具体的Skill定制。这使得它们非常适合由AI生成初稿，由你来定制和审核。

更重要的是，让AI设计测试方案本身就是一种“元测试”的练习：你正在使用AI来帮助测试AI。这种能力——知道如何向AI提问以获得高质量的测试设计——是AI时代设计师的元能力。

6.2.2 AI生成测试方案的提示词模板

以下是一个通用的测试方案生成提示词。请将方括号中的内容替换为你的Skill的具体信息。

```
我需要为我的设计研究AI Skill设计一套测试方案。我的Skill是：[一句话描述，如“一个将用户访谈文本自动聚类为主题并生成摘要的Skill”]。输入是：[输入类型和格式]，输出是：[输出类型和结构]。

请帮我生成一份测试方案初稿，包含以下内容：

1. **测试目标**：用一段话说明本次测试要验证什么（例如：验证准确性是否达到90%以上、验证与纯人工相比效率提升50%以上）。

2. **测试用例设计**：设计10-15个测试用例，覆盖：
   - 正常场景：有代表性的数据输入（5-7个）
   - 边界场景：极端短/长输入、有歧义的表达、混合语言（3-5个）
   - 异常场景：错误格式、空输入、超大规模（2-3个）
   对每个测试用例给出：输入内容的示例、预期行为的文字描述、难度评级（easy/medium/hard）。

3. **测试数据集说明**：总共需要多少条数据？数据的来源是什么（真实/模拟）？如果模拟，需要模拟哪些特征分布？

4. **准确性测试的具体方法**：参考答案如何获取（人工标注/AI预标注+人工修正）？逐条比对的方式是什么？准确率计算采用什么指标（如完全匹配、部分匹配、语义相似度）？

5. **一致性测试的具体方法**：重测的步骤是什么（间隔多久？是否改变随机种子？）？一致性如何量化（如Kappa系数、余弦相似度）？

6. **自动化测试脚本框架**：请用Python写一个测试脚本框架，实现：读取测试数据 → 调用Skill的分析函数 → 与参考答案比对 → 生成测试摘要。

7. **测试报告模板**：一份包含了上述所有测试维度的报告结构（Markdown格式）。
```

6.2.3 审核AI生成的测试方案

AI生成的测试方案通常质量较高，但仍需人工审核以下关键点：

- **测试用例是否真正覆盖了你的Skill的功能范围？** AI可能遗漏某些你Skill特有的功能模块。例如，如果你的Skill有人工修正节点，测试方案中应包含对该节点交互的测试用例。
- **边界和异常用例是否合理？** AI有时会编造不太可能在真实场景中出现的边界情况（如“输入一个包含10000个字符的文本”），你需要判断这些是否真的有必要测试。
- **参考答案的标注标准是否可操作？** 如果标准模糊（如“主题名称应该准确”），实际标注时会混乱。你需要将标准具体化（如“主题名称应能在3-5个字内概括该组80%以上条目的核心内容”）。
- **测试脚本框架是否与你的代码兼容？** 检查函数名、参数类型、返回值结构是否匹配。不匹配的地方需要在后续修改。

审核后的方案将成为第6.3节测试准备工作的指南。

6.3 测试数据的生成与基线构建

6.3.1 测试数据的质量标准

测试数据是测试的“原材料”，其质量直接决定了测试结论的有效性。一条合格的测试数据应同时满足以下五个标准：

1. **代表性**：测试数据应反映Skill预期使用场景中的数据分布。如果Skill处理的是用户反馈，测试数据中应包含各种情感倾向、各种话题、各种表达风格（简洁、啰嗦、专业、口语）。不要只生成“容易判断的”数据——那会让准确率虚高。
2. **多样性**：测试数据必须覆盖不同的难度级别。建议按“容易:中等:困难 = 3:5:2”的比例分配。容易的条目用于验证基础能力，困难的条目用于揭示能力边界。
3. **独立性**：测试数据不应来自开发阶段使用过的数据集。用同一批数据进行开发和测试会导致过拟合——Skill可能“记住”了开发数据中的模式，在测试时表现虚高。
4. **规模适中**：测试数据集不必太大。15-30条核心测试数据通常足够发现系统性的问题。如果数量太多，人工标注参考答案的成本会过高；如果数量太少，统计结果可能不稳定。
5. **可标注性**：每条测试数据必须能够被赋予一个明确的参考答案（或至少一个可评判的标准）。对于难以达成共识的模糊案例，应排除出核心测试集，或标注为“主观判断”并在报告中单独讨论。

6.3.2 用AI生成模拟测试数据

对于大多数设计研究项目，真实的、带有标准答案的测试数据很难获取。因此，用AI生成模拟测试数据是一种高效且可控的策略。但和开发阶段生成模拟数据不同，测试数据的质量要求更高——你需要明确指定数据的分布、难度层级和标注要求。

以下是一个高质量的测试数据生成提示词：

```
请为我的[Skill名称]生成一组测试数据集。我的Skill处理[输入类型]，输出[输出类型]。

**数据要求**：
- 生成15条测试条目
- 情感/主题分布：[例如：5条正面、5条负面、5条中性]
- 难度分布：约5条easy（表面含义清晰，判断直接）、7条medium（需要语境理解）、3条hard（包含反讽、比喻或专业术语）
- 来源多样性：模拟不同用户类型/场景（如新手用户、专家用户、临时用户）
- 语言自然，模拟真实用户的表达风格，避免过于工整或模板化

**标注要求**：
对每条测试数据，同时生成以下标注信息：
- sentiment：正面/负面/中性
- key_topics：1-3个关键词或短语
- difficulty：easy/medium/hard
- annotation_rationale：一句话说明为什么这样标注（作为参考答案的依据）

**输出格式**：
输出为一个JSON列表，每个元素是一个字典，包含 id、text、sentiment、key_topics、difficulty、annotation_rationale。

**注意**：这些数据将用于系统测试，请不要包含我开发阶段可能使用过的类似内容。数据需要有足够的真实感和区分度。
```

生成后的必要审查：

AI生成的测试数据可能存在以下问题，必须在正式使用前进行人工审查：

问题类型	表现	应对方法
同质化	多条数据使用相似的句式和措辞	快速扫读，删除模板化明显的条目，重新生成
难度虚标	AI标注为hard的条目实际并不困难	逐条确认难度，必要时重新标注
标注错误	AI自带的参考答案与文本实际含义不符	随机抽取30%条目，由组员独立重新标注，比对一致性
内容重复	多条数据实质上表达了相同的含义	逐条比对语义，删除重复条目

完成审查和修正后，测试数据集才可投入使用。

6.3.3 参考答案的构建

参考答案是准确性测试的“标尺”。以下三种方式可根据你的资源情况选择：

方式一：人工标注（最可靠）

你和组员作为“方法专家”，对测试数据进行逐条标注。如果多个组员一起标注，可以计算标注者间信度（Inter-rater Reliability）——这本身就是一个有价值的数据点，反映了该方法在人类研究者中的一致性水平。人工标注适合核心的10-15条测试数据。

方式二：AI预标注 + 人工修正（平衡方案）

让一个与你的Skill使用不同模型（如GPT-4）的AI实例对测试数据进行预标注，然后由你的组员审核并修正。这种方式结合了AI的效率与人的判断力。

> 请使用GPT-4模型（temperature=0）对以下测试数据进行标注。每条数据标注sentiment、key_topics和difficulty。请给出标注依据。这些标注将由人类审核，请尽量准确。

方式三：多AI互评（仅适用于简单维度）

对于字面准确性这类相对客观的判断，可以让多个AI模型分别标注，取它们一致的结果作为参考答案，不一致的条目提交人类仲裁。这种方式适合快速构建大规模测试集的参考答案，但风险是可能放大模型间的共同偏见。

6.3.4 基线的选择与构建

要衡量你的Skill带来了多少增益，你需要一个基线——没有你的Skill，这项任务通常是怎么完成的。以下是四种可行的基线选择：

基线类型	描述	适用场景	构建工作量
纯人工	一位研究者用传统方式完成任务	验证Skill的效率增益	高（需实际执行）
通用AI工具	使用ChatGPT/Claude对话界面手动完成	验证Skill的专用性增益	中
简单脚本基线	最简版分析脚本（零样本分类）	验证复杂处理链的价值	低
历史版本	你的Skill的MVP版本	验证迭代的改进效果	低（如有日志）

对于大多数课程项目，建议采用**简单脚本基线**——用最少的提示词工程和预处理直接调用API，作为比较的参照。这样可以量化"精心设计的处理链"相比"粗暴调API"带来了多少提升。

构建简单脚本基线的Vibe Coding描述：

> 请写一个简单的基线脚本 `baseline.py`，功能如下：
>
> - 读取测试数据中的文本字段
> - 对每条文本，调用DeepSeek API（temperature=0）进行[情感分析/主题提取/分类]，使用最简提示词："请分析以下文本的情感极性，只输出正面/负面/中性。"
> - 不进行任何预处理、后处理或结果解析优化
> - 将输出保存到 `baseline_results.json`
>
> 这个基线将用于与我的完整Skill进行对比。

6.4 测试执行：定量与定性双轨收集

6.4.1 定量测试的自动化执行

在测试准备就绪后，定量测试的执行应尽可能自动化，以减少手工操作误差并提高效率。以下是一个完整的自动化测试脚本框架，可通过Vibe Coding生成：

> 请写一个自动化测试运行脚本 `run_tests.py`，功能如下：
>
> 1. 从 `test_data.json` 读取测试用例（包含id、输入文本、参考答案字段）
> 2. 对每条测试用例，调用我的Skill的分析函数（从 `skill.py` 导入）
> 3. 记录每条的处理耗时（使用 `time.time()` 包裹调用）
> 4. 将每条的输出保存到 `test_results/result_{id}.json`
> 5. 对每条输出，与参考答案进行比对：
>   - 对于分类字段（如sentiment）：直接比较是否相等
>   - 对于生成字段（如key_topics）：调用LLM裁判判断语义一致度（见6.4.2）
> 6. 处理过程中显示进度条（使用tqdm）
> 7. 测试完成后，生成汇总表格 `test_summary.csv`，包含：id、耗时、各字段比对结果（通过/不通过）、备注
> 8. 同时生成一个 `test_summary.json`，包含整体统计：平均耗时、各分类准确率、混淆矩阵

执行前的预检：

在运行完整测试之前，先用3条测试数据进行预检，确认：

- Skill能正常加载并处理这些数据
- 输出格式与预期一致
- 比对逻辑能正确执行

预检通过后再运行全量测试。

6.4.2 LLM-as-Judge：用AI评判生成式输出

对于生成式输出（如主题命名、摘要、Persona叙事），无法用简单的字符串匹配来判断是否正确。LLM-as-Judge是一种可行的解决方案：使用一个强大的、与你的Skill不同的模型作为"裁判"，评判生成内容的语义质量。

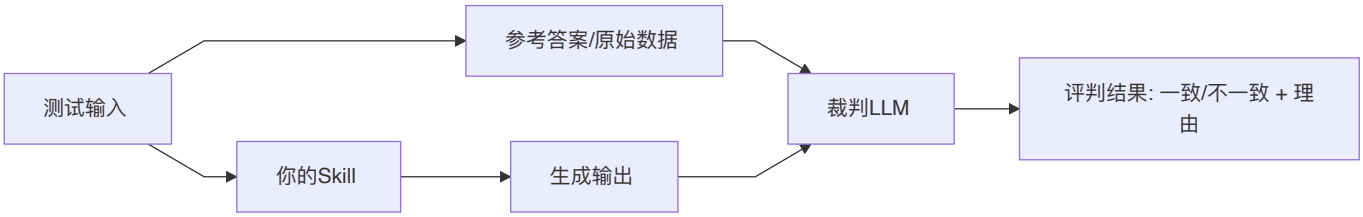


图6-2 LLM-as-Judge裁判流程

LLM-as-Judge的Vibe Coding实现：

> 请写一个 `llm_judge` 函数，用于评估Skill生成的文本输出质量。函数签名：
> `def llm_judge(generated_text, reference_text, criteria) -> dict`
>
> 参数说明：
>
> - generated_text: Skill生成的文本
> - reference_text: 参考答案或原始数据
> - criteria: 评判标准，如"判断生成的摘要是否覆盖了参考文本中的关键信息"
>
> 函数内部调用裁判模型API（使用GPT-4或Claude，temperature=0），构造如下提示词：
>
> ```
> 你是一个严格的评审专家。请判断以下【生成文本】是否准确地反映了【参考答案】中的关键信息。
>
> 【生成文本】：{generated_text}
> 【参考答案】：{reference_text}
> 【评判标准】：{criteria}
>
> 请输出一个JSON，包含：
> - "judgment": "一致" / "部分一致" / "不一致"
> - "confidence": 0.0-1.0
> - "reason": 一句话理由
> ```
>
> 返回解析后的字典。如果裁判调用失败，返回 `{"judgment": "error", "reason": "裁判API调用失败"}`。

LLM-as-Judge的局限与应对：

- **裁判偏见**：裁判模型可能偏爱特定风格（如更长的回答、更正式的措辞），这不一定是“正确”。应对：在评判标准中明确要求忽略风格差异，只关注事实一致性。
- **循环依赖**：如果用同一个模型既作为被测试对象又作为裁判，可能会产生自我偏好。应对：选择与被测试模型不同的裁判模型（如Skill使用DeepSeek，裁判使用GPT-4）。
- **成本**：对每条输出调用一次裁判API会增加时间和费用。应对：只对生成字段（而非全部分类字段）使用裁判，且只对核心测试集使用。

6.4.3 定性体验测试的执行

定量测试回答“Skill做得对不对”，定性体验测试回答“人觉得好不好用”。定性测试建议邀请2-4名外部被试（最好是未参与开发的设计专业同学），每位被试完成一个标准任务。

标准任务设计：

为被试设计一个具体、可完成的任务。例如：“请使用这个工具分析附件中的5条用户反馈，找出其中最重要的3个问题主题，并生成一份简要报告。”任务说明必须标准化——对被试说的话要一致，不要给不同被试不同的引导。

测试前的指导语：

> 感谢你参与本次测试。你将要测试的是一个[描述Skill功能]的工具。请按照任务说明独立完成操作。**你测试的是这个工具，不是你的能力**。遇到任何问题都是工具的问题，请大声说出你的困惑（例如：“我不知道下一步该点什么”“这个结果我看不懂”）。我会在旁边观察，但在你主动求助之前不会干预。

观察与记录：

在测试过程中，观察并记录以下行为：

- 被试在哪个环节停下来不知道下一步该做什么？（记录界面位置和停留时间）
- 被试是否有说出困惑（出声思维）？困惑的具体内容是什么？
- 被试是否在没有提示的情况下自主找到了解决方案？（如果找到了，说明界面具有可探索性）
- 被试是否误操作了（如点击了错误的按钮）？

测试后的数据收集：

1. **SUS量表**（System Usability Scale）：10个标准条目，每个条目1-5分。SUS总分计算公式：(Q1+Q3+Q5+Q7+Q9 - 5) + (25 - (Q2+Q4+Q6+Q8+Q10))，再乘以2.5得到0-100分。68分是平均水平，80分以上为优秀。
2. **自定义体验条目**（1-5分）：
    - “我对AI输出的分析结果感到信任”
    - “我能够理解AI为什么给出这样的结论”
    - “我感受到了对分析过程的控制感”
    - “下次有类似任务，我愿意再次使用这个工具”
3. **半结构化访谈**（5-10分钟）：
    - “整个过程中最让你困惑的是哪个环节？”
    - “如果下次你需要做类似的分析，你愿意用这个工具吗？为什么/为什么不？”
    - “你觉得AI给出的分析结果，你在什么情况下最信任？什么情况下最不信任？”

- “如果有一个魔法可以改变这个工具的一件事，你会改变什么？”

记录访谈要点，特别是与你的预期不一致的发现。

6.5 测试数据分析与报告撰写

6.5.1 定量数据的AI辅助分析

测试执行完成后，你将面对一堆原始数据——时间戳、准确性分数、用户评分、访谈记录。逐条手工分析不仅耗时，而且容易遗漏模式。AI可以协助进行第一轮的数据处理和统计分析。

数据清理与准备：

> 我有一批测试结果原始数据，请帮我写一个数据清理脚本。读取 `test_summary.json`，执行以下操作：
>
> - 检查每条记录是否包含所有必需字段；缺失关键字段的记录标记为不完整
> - 检查数值字段的范围：耗时不能为负数，准确率在0-1之间
> - 移除处理失败的测试条目（如API调用返回错误），导出到 `error_log.json`
> - 输出清理后的数据集和清理报告

统计分析的自然语言生成：

> 基于清理后的测试数据，请生成以下统计摘要（用自然语言段落表述，不是纯数字列表）：
>
> 1. **准确性摘要**：整体准确率是多少？按难度层级（easy/medium/hard）分解的准确率分别是多少？哪个类别最容易被误判（混淆矩阵的主要错误模式）？
> 2. **效率摘要**：平均端到端耗时是多少？最慢的20%用例有什么共同特征（如输入长度、难度）？
> 3. **一致性摘要**：重测一致率是多少？不同测试用例间的一致性标准差是多少？
> 4. **基线对比**：与简单脚本基线相比，你的Skill在准确率和耗时上分别有怎样的差异？

6.5.2 定性数据的主题提取

对于访谈记录和开放式问卷回答，可以使用AI进行主题分析（Thematic Analysis），但必须由你进行最终审核。

> 以下是3-5位被试的访谈记录摘要（每段200-300字）。请帮我：
>
> 1. 提取每个被试提到的主要困惑点（列出具体问题）
> 2. 提取所有被试共同提到的正面评价（至少2条）
> 3. 提取所有被试共同提到的负面评价或改进建议（至少3条）
> 4. 按出现频率对改进建议排序
> 5. 生成一段200字左右的体验测试总结，突出关键发现

人工审核要点：AI提取的主题可能遗漏某些微妙但重要的细节，也可能将个别被试的独特观点误判为普遍现象。你需要仔细阅读原始访谈记录，确认AI的提取是否准确、完整。

6.5.3 测试报告的完整结构

一份完整的设计研究AI Skill测试报告应包含以下章节。建议使用AI生成初稿，然后由小组审核修改。

```
# [Skill名称] 测试报告

**版本**: v1.0
**测试日期**: YYYY-MM-DD
**测试负责人**: [姓名]

## 1. 测试概览
- 测试目标：本次测试要回答的核心问题（例如：验证准确率≥85%，验证与纯人工相比效率提升≥50%）
- 测试范围：覆盖的功能模块、未覆盖的功能模块及原因
- 测试数据集：数据来源、规模、难度分布
- 测试参与者：定量测试（自动化）、定性测试（x名外部被试）

## 2. 测试方法与指标
- 各维度的度量指标定义
- 基线说明（简单脚本/纯人工/前版本）
- 测试环境（硬件、软件、API版本）

## 3. 准确性测试结果
- 整体准确率：XX%
- 按难度分解：easy XX%，medium XX%，hard XX%
- 混淆矩阵（如有）：展示最容易被误判的类别对
- 错误分析：主要的错误类型和典型错误案例（附2-3个例子）
- **结论**：准确性是否达到预期目标?

## 4. 一致性测试结果
- 重测一致率：XX%
- 边界稳定性：处理异常输入的成功率 XX%
- **结论**：一致性水平是否可接受?

## 5. 效率测试结果
- 端到端平均耗时：XX秒
- 与基线对比：相比简单脚本耗时 ±XX%
- 各环节耗时分布（如预处理、AI调用、后处理）
- **结论**：效率增益是否显著?

## 6. 健壮性测试结果
- 各类异常输入的通过率表格
- 典型的错误处理表现（附截图或日志摘要）
- **结论**：健壮性是否达到生产可用标准?

## 7. 体验测试结果
- SUS总分：XX分（百分位：XX%）
- 自定义条目均分（信任感、控制感、理解度、使用意愿）
- 用户反馈的主要困惑点（引用典型访谈原话）
- 用户最欣赏的设计点
- **结论**：体验是否满足用户预期?

## 8. 协同增益分析（可选）
- 人机协同 vs AI单独 vs 人类单独的质量对比
- 增益量化（如质量提升XX%）
- **结论**：协同是否产生正增益?

```

```
## 9. 改进计划
- 基于测试发现，按优先级列出的改进项（P0/P1/P2）
- 每项改进的目标和预期工作量
- 计划在第7周完成的优化
```

6.5.4 从测试发现到改进计划

测试报告的价值不仅在于“证明了什么”，更在于“接下来做什么”。基于测试发现，每组应制定一个优先级明确的改进计划。

优先级	定义	示例
P0（必须修复）	影响核心功能或导致测试无法完成的问题	API调用频繁失败、输出格式解析错误、严重的准确性缺陷（<60%）
P1（优先处理）	显著影响用户体验或准确率的问题	某类输入准确率偏低、被试普遍困惑的操作节点、效率低于预期
P2（有余力时优化）	锦上添花的改进	界面微调、额外的输出格式、更丰富的图表

改进计划中的每一项都应具体、可验证。例如：

- ❌ 不好的写法：“提高准确性”
- ✅ 好的写法：“针对hard难度测试用例中‘反讽’类误判，优化分析提示词，增加反讽识别示例，目标将该类准确率从40%提升到70%”

6.6 本章小结

本章完成了从“开发”到“验证”的关键跨越。我们系统性地探讨了设计研究AI工具的测试方法论，其核心要点可概括为以下六条：

第一，测试AI Skill不同于测试传统软件。由于输出不确定、正确答案模糊、错误表现为偏差而非崩溃，我们需要一套新的测试框架——五维测试框架（准确性、一致性、效率、健壮性、体验）加上协同增益分析。

第二，AI可以成为测试的助手。从测试方案设计、测试数据生成，到自动化脚本编写和LLM-as-Judge裁判，AI能够大幅降低测试的工作量。但人工审核仍然是不可替代的——你对测试数据质量的判断、对参考答案的确认、对测试结论的诠释，是AI无法代劳的专业判断。

第三，测试数据的质量决定测试结论的有效性。测试数据必须具有代表性、多样性、独立性、适中的规模和可标注性。用AI生成模拟数据时，必须进行人工审查，剔除同质化、难度虚标和标注错误的条目。

第四，定量测试应尽可能自动化。通过编写自动化测试脚本，可以批量运行测试、记录耗时、比对结果、生成统计摘要。LLM-as-Judge技术可以帮助评估生成式输出的语义质量，但需注意裁判偏见和成本控制。

第五，定性体验测试揭示量化数据无法捕捉的洞察。SUS量表提供整体可用性评分，半结构化访谈揭示具体的困惑点和改进方向。观察被试的操作行为往往比听他们事后评价更能发现问题。

第六，测试报告的终点是改进计划。将测试发现转化为优先级明确的P0/P1/P2改进项，并关联到具体的优化行动。测试不是项目的结束，而是持续迭代的起点。

在第7章中，我们将基于测试报告的发现，完成最后一轮优化，并将Skill的核心子功能封装为API，准备最终的课程交付。

思考与练习

一、概念理解题

1. 请解释测试AI Skill与传统软件测试的三个本质差异，并分别说明这些差异对测试方案设计的影响。
2. 五维测试框架中的“准确性”包含哪三个层次？为什么在设计研究Skill中需要区分这些层次？
3. 什么是“协同增益”？如何设计实验来测量人机协同是否产生了正增益？
4. LLM-as-Judge有哪些优势和局限？在什么情况下应该使用它，什么情况下应该避免？

二、分析应用题

5. 以下是一个情感分析Skill的测试结果片段，请基于数据分析该Skill的主要问题，并提出改进方向。

难度	准确率	主要错误模式
easy	95%	无显著模式
medium	78%	将“略微负面”误判为“中性”（占错误的60%）
hard	45%	将反讽误判为正面（占错误的70%）

6. 假设你的Skill在用户测试中SUS得分仅为55分（低于平均水平）。你收到了以下三条用户反馈：(a)“我不知道上传文件后还需要点击哪里才能开始分析”；(b)“AI生成的标签有时很奇怪，但我不知道怎么改”；(c)“处理进度条一直不增长，我以为死机了”。请为每条反馈设计一个具体的改进方案，并转换为Vibe Coding修改指令。

三、实践与反思题

7. 基于你的Skill，完成以下测试准备工作，并提交相关文件：
    - 使用AI生成15条测试数据（包含标注）
    - 人工审查并修正其中至少5条
    - 编写自动化测试脚本，运行至少10条测试用例
    - 记录测试结果，填入测试报告模板的“准确性”和“效率”部分
8. 邀请2名外部同学作为被试，执行定性体验测试。提交以下材料：
    - 标准任务说明
    - 测试过程中的观察笔记（至少记录3个困惑点）
    - SUS量表原始数据及计算结果
    - 半结构化访谈记录摘要
    - 基于测试发现，列出3条P1级别的改进计划
9. **个人反思**（500字）：在本次测试过程中，你最大的意外发现是什么（例如：某个你原本以为很准的功能实

际准确率很低，或者某个你忽略的体验问题被多位被试提到）？这个发现如何改变了你对“设计研究AI工具评估”的理解？

参考文献

[1] Amershi, S., et al. (2019). Guidelines for human-AI interaction. *Proceedings of CHI 2019*.

[2] Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. *Proceedings of ACL 2020*, 4902-4912.

[3] Zheng, L., et al. (2023). Judging LLM-as-a-judge with MT-bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*.

[4] Brooke, J. (1996). SUS: A quick and dirty usability scale. *Usability Evaluation in Industry*, 189-194.

[5] Sauro, J., & Lewis, J. R. (2016). *Quantifying the user experience: Practical statistics for user research*. Morgan Kaufmann.

[6] Karpathy, A. (2024). Vibe coding: A new way to program. *Personal blog / X*.

本章引用统计数据：本章全文约 18,500 字。